Operating Room Scheduling Optimization

A datadriven approach to solving one of healthcare "'s most persistent operational headaches — getting the right surgery into the right room at the right time.

What This Project Is About?

Anyone who”s spent time in a hospital setting knows how chaotic the surgery floor can get. You”ve got multiple operating rooms, a long list of scheduled procedures, different surgery durations, and a very real cost to getting the timing wrong — delayed patients, idle rooms, frustrated staff.

This project tackles that problem headon using “Constraint Programming” and “Google ORTools” to build an optimized surgery schedule. The goal is simple: assign surgeries to available rooms in a way that avoids overlaps, minimizes completion time, and makes better use of the resources already available.

Dataset

_The dataset (“or_optimization_dataset.csv”) contains realworldstyle surgical records with the following fields:

Column	Description
“hadm_id”	Unique hospital admission ID
“procedure_type”	Type of surgery (Cardiac, Neuro, Orthopedic, General)
“scheduled_start_time”	Originally scheduled date
“scheduled_duration”	Planned duration (minutes)
“actual_duration”	How long it actually took
“emergency_flag”	Whether it was an emergency case
“acuity_score”	Patient severity score
“or_room_id”	Operating room assigned (OR1, OR2, OR3)
“surgeon_id”	Surgeon on the case
“equipment_id”	Equipment used

The dataset covers “130 surgical cases” from January 2025, across 4 procedure types and 3 operating rooms.

How It Works

1. Data Preparation

Surgery durations are extracted and cleaned. Each surgery is treated as a job that needs a start time, an end time, and a room assignment.

2. Optimization Model (ORTools CPSAT)

We build a Constraint Programming model where:

- Each surgery gets a “start time” and “end time” variable
- Surgeries are assigned to “one of three rooms” (OR1, OR2, OR3)
- The model enforces “no overlapping surgeries” in the same room
- The objective is to “minimize the overall makespan” (total completion time)

3. Schedule Output

To find the best possible outcome were a clean schedule with each surgery's room, start time, and end time.

A sample from the output (`schedule_output.csv`):

Result

surgery	room	start	end
0	OR2	1046	1208
1	OR2	0	239
2	OR1	720	872

Times are in minutes from the start of the scheduling window.

Project Structure

task.ipynb (Main Jupyter notebook with the full solution)

or_optimization_dataset.csv (Source dataset)

original_data.csv (Raw data)

projectdataset.csv (Processed dataset used in modeling)

schedule_output.csv (Final optimized schedule)

orprojectoutput.pbix (dashboard file)

Report.pdf (Project report and findings)

README.md (You're reading it)

Tech Stack

"Python" Core programming language

"Google ORTools" Constraint Programming solver

"Pandas" Data loading and manipulation

"Matplotlib" Basic visualization

"Power BI" –Dashboard creation

"Jupyter Notebook" Development environment

Running the Notebook

bash

pip install ortools pandas matplotlib

"Steps:"

1. Clone or download the repository

2. Place all CSV files in the same directory as `task.ipynb.`

3. Open and run `task.ipynb` in Jupyter

4. The optimized schedule will be saved as `schedule_output.csv.`

What Could Be Better

This is a solid first version, but there's room to grow. A few things worth adding in future iterations:

“Emergency surgery handling” — right now, the model treats all cases equally; real hospitals don't

“Surgeon and staff availability” — a surgeon can't be in two rooms at once

“Equipment constraints” — some procedures share equipment that can't be double-booked

“Realtime rescheduling” — adjusting on the fly when cases run over

“ML-based duration prediction” — using historical data to predict how long a surgery will actually take (not just what's scheduled)